CS-375 Assignment 4 Report:

- **Student Name: Alex Eskenazi**
- **Student ID: B00944943**

- B.1 Graph - Done
- B.2 Linear Programming - Done
- B.3 Simplex (Extra Credit) - Done

[Part B]: Programming Part (26%)

B.1 Graph Algorithm (16%)

Suppose a CS curriculum consists of n courses, all of them mandatory. The prerequisite graph G has a node for each course, and an edge from course v to course w if and only if v is a prerequisite for w. Find an algorithm that works directly with this graph representation, and computes the minimum number of semesters necessary to complete the curriculum (Assume that a student can take any number of courses in one semester).

Using following example to justify your answer:
The CS Department requires fifteen one-semester courses with the prerequisites shown below:

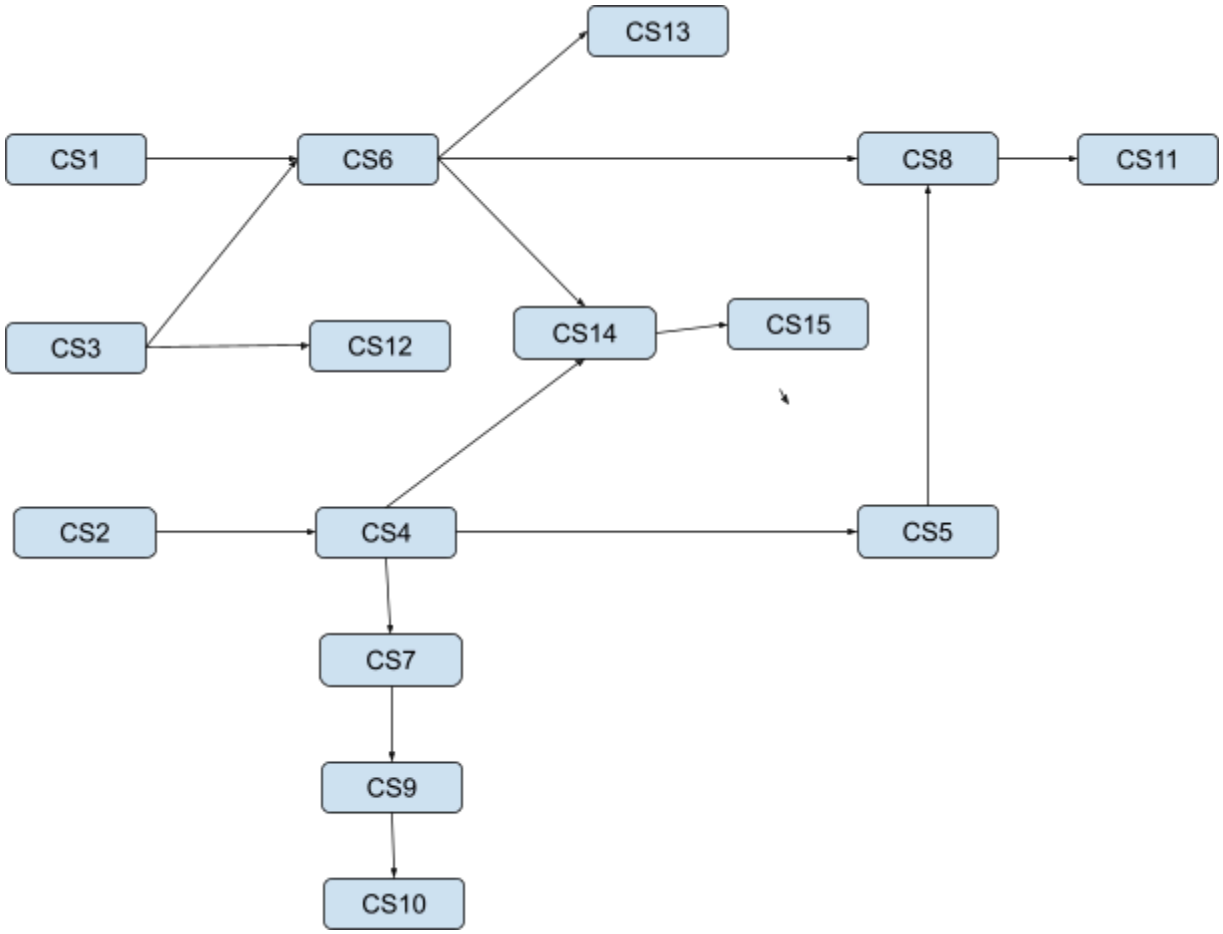
cs1
cs2
cs3
cs4 requires cs2
cs5 requires cs4
cs6 requires cs1 and cs3
cs7 requires cs4
cs8 requires cs5 and cs6
cs9 requires cs7
cs10 requires cs9
cs11 requires cs8
cs12 requires cs3
cs13 requires cs6
cs14 requires cs4 and cs6
cs15 requires cs14

Your task is to determine the minimum number of semesters needed to finish the degree.

(Hint: Represent the courses and their prerequisites as a DAG. Using appropriate data structure to represent the graph).

Please provide:

1. Manually plot the DAG (3%)



2. Explain the algorithm that you are going to implement by the Pseudo-code, and indicate the minimum number of semesters necessary to finish the degree. Indicate the time complexity and space complexity (3%)

   The algorithm uses topological sorting with level tracking to find the minimum number of semesters needed. Each course is assigned to the earliest possible semester based on when all its prerequisites are completed.

   Pseudo Code

   Input: DAG G(V, E) where V are courses and E are prerequisite edges (v → w)

   Initialize:

   semester[i] = 0 for each course i in G

```
Get topological order:
    topoOrder = TopologicalSort(G)

For each course c in topoOrder:
    maxPrereqSemester = 0
    For each prerequisite p of c:
        maxPrereqSemester = max(maxPrereqSemester, semester[p])
    semester[c] = maxPrereqSemester + 1

Return max(semester[i]) for all i
```

Pseudo for TopologicalSort(G)

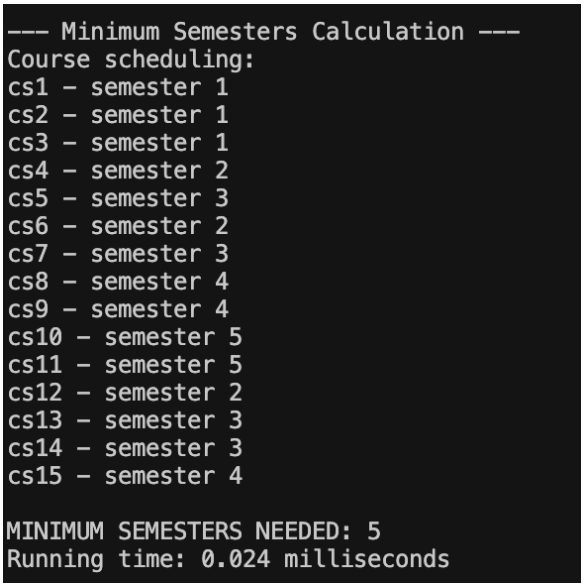
```
Initialize indegree[i] = number of prerequisites for course i
  Queue = all courses with indegree = 0
  result = []

  While Queue not empty:
      remove node u from Queue
       add u to result
       for each neighbor v of u:
            indegree[v] -= 1
            if indegree[v] == 0:
                add v to Queue

      Return result
```

Complexity Analysis

Time Complexity:
O(V + E) , because both topological sorting and semester computation iterate through all nodes and edges once.

Space Complexity:
O(V + E), adjacency list, indegree array, and semester array.

3. Write and run your program to print out the result for verification (i.e., minimum number of semesters, and print out the running time) (10%)



Minimum number of semesters necessary: 5

Semester Schedule:
- Semester 1: cs1, cs2, cs3
- Semester 2: cs4, cs6, cs12
- Semester 3: cs5, cs7, cs13, cs14
- Semester 4: cs8, cs9, cs15
- Semester 5: cs10, cs11

The running time was 0.024 milliseconds

B.2. Linear Programming (10%)

Design and implement a linear programming algorithm to solve the following minimum cost problem

The liquid portion of a diet is to provide at least 300 calories, 36 units of vitamin A, and 90 units of vitamin C daily. A cup of dietary drink X provides 60 calories, 12 units of vitamin A, and 10 units of vitamin C. A cup of dietary drink Y provides 60 calories, 6 units of vitamin A, and 30 units of vitamin C. Now, suppose that dietary drink X costs $0.12 per cup and drink Y costs $0.15 per cup. How many cups of each drink should be consumed each day to minimize the cost and still meet the stated daily requirements?

Print out the minimum cost, and the number cups of drink X and number of cups of drink Y; Print out the running time.

See the project code. These are the results:

- === OPTIMAL SOLUTION ===
- Number of cups of drink X: 3
- Number of cups of drink Y: 2
- Minimum cost: $0.66
-
- Nutrient totals provided:
- Calories: 300 (required: >= 300)
- Vitamin A: 48 (required: >= 36)
- Vitamin C: 90 (required: >= 90)
- Running time: 0.011 milliseconds

B.3. [Optional Extra point 10%]

To answer the Question #9 (Part A), implement the Simplex Algorithm and display the results by your program, and print out the running time.

```
--- Iteration 3 ---
Entering variable: x3
Ratio test:
  Row 0: 8/5 = 1.6
  Row 2: 9/4 = 2.25
  Row 3: 5/2 = 2.5
Leaving variable: s1
Pivot element: 5
Making pivot element 1 by dividing row 0 by 5
Making row 1 element 0 using factor -3
Making row 2 element 0 using factor 4
Making row 3 element 0 using factor 2
Making row 4 element 0 using factor -5

After pivot operations:
    x1      x2      x3      s1      s2      s3      s4      RHS
-----------------------------------------------------------
s1 0    0       1       0.2     -0.8    0       0.2     1.6
s2 0    1       0       0.6     2.6     0       -1.4    20.8
s3 0    0       0       -0.8    0.2     1       0.2     2.6
s4 1    0       0       -0.4    -0.4    0       0.6     1.8
P  0    0       0       1       6       0       1       268
-----------------------------------------------------------

=== OPTIMAL SOLUTION ===
x1 = 1.8
x2 = 20.8
x3 = 1.6
Maximum P = 268

Running time: 0.137 milliseconds
==================================
```

CS 375 Assignment 4 - Part B.3
Simplex Algorithm for Question #9
==========
Solving using Simplex Method...
Problem: Maximize P = 20x1 + 10x2 + 15x3
Initial table:

	x1	x2	x3	s1	s2	s3	s4	RHS
s1	3	2	5	1	0	0	0	55
s2	2	1	1	0	1	0	0	26
s3	1	1	3	0	0	1	0	30
s4	5	2	4	0	0	0	1	57
P	-20	-10	-15	0	0	0	0	0

--- Iteration 1 ---
Entering variable: x1

Ratio test:
  Row 0: 55/3 = 18.3333
  Row 1: 26/2 = 13
  Row 2: 30/1 = 30
  Row 3: 57/5 = 11.4
Leaving variable: s4
Pivot element: 5
Making pivot element 1 by dividing row 3 by 5
Making row 0 element 0 using factor 3
Making row 1 element 0 using factor 2
Making row 2 element 0 using factor 1
Making row 4 element 0 using factor -20

After pivot operations:

	x1	x2	x3	s1	s2	s3	s4	RHS
s1	0	0.8	2.6	1	0	0	-0.6	20.8
s2	0	0.2	-0.6	0	1	0	-0.4	3.2
s3	0	0.6	2.2	0	0	1	-0.2	18.6
s4	1	0.4	0.8	0	0	0	0.2	11.4
P	0	-2	1	0	0	0	4	228

--- Iteration 2 ---
Entering variable: x2
Ratio test:
  Row 0: 20.8/0.8 = 26
  Row 1: 3.2/0.2 = 16
  Row 2: 18.6/0.6 = 31
  Row 3: 11.4/0.4 = 28.5
Leaving variable: s2
Pivot element: 0.2
Making pivot element 1 by dividing row 1 by 0.2
Making row 0 element 0 using factor 0.8
Making row 2 element 0 using factor 0.6
Making row 3 element 0 using factor 0.4
Making row 4 element 0 using factor -2

After pivot operations:

	x1	x2	x3	s1	s2	s3	s4	RHS
s1	0	0	5	1	-4	0	1	8
s2	0	1	-3	0	5	0	-2	16
S3	0	0	4	0	-3	1	1	9

s4	1	0	2	0	-2	0	1	5
P	0	0	-5	0	10	0	1.97215e-31	260

--- Iteration 3 ---
Entering variable: x3
Ratio test:
  Row 0: 8/5 = 1.6
  Row 2: 9/4 = 2.25
  Row 3: 5/2 = 2.5
Leaving variable: s1
Pivot element: 5
Making pivot element 1 by dividing row 0 by 5
Making row 1 element 0 using factor -3
Making row 2 element 0 using factor 4
Making row 3 element 0 using factor 2
Making row 4 element 0 using factor -5

After pivot operations:

	x1	x2	x3	s1	s2	s3	s4	RHS
s1	0	0	1	0.2	-0.8	0	0.2	1.6
S2	0	1	0	0.6	2.6	0	-1.4	20.8
s3	0	0	0	-0.8	0.2	1	0.2	2.6
s4	1	0	0	-0.4	-0.4	0	0.6	1.8
P	0	0	0	1	6	0	1	268

=== OPTIMAL SOLUTION ===
x1 = 1.8
x2 = 20.8
x3 = 1.6
Maximum P = 268

Running time: 0.267 milliseconds
==========